DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA0603 – DESIGN AND ANALYSIS OF ALGORITHMS

Assignment Report

Analytical Study and Implementation of Brute Force Algorithms for Selection Sort and Sequential Search

Name: Moshitha Kolanjiyappan (192421205)

Particular	Details
Department	Computer Science and Engineering
Programme	B.E/B.Tech – Computer Science and Engineering
Course Code & Name	CSA0603 – Design and Analysis of Algorithms
Academic Year / Batch	2026 – Regular
Faculty Name	Dr. P. Solainayagi
Assignment Title	Analytical Study and Implementation of Brute Force Algorithms for Selection Sort and Sequential Search
Date of Issue	01-09-2026
Date of Submission	02-09-2025 (verify with faculty; likely 02-09-2026)
Maximum Marks	100
Course Outcome	CO2 – Apply Brute Force, Searching, Sorting, Closest Pair, Convex Hull
Bloom's Taxonomy	L4 – Analyse; L5 – Evaluate; L6 – Create
SDG Mapping	SDG 4 – Quality Education; SDG 9 – Industry, Innovation and Infrastructure

1. Problem Understanding and Formulation

The objective of this assignment is to study and implement two brute-force algorithms: Selection Sort for sorting and Sequential Search for searching. The given input array is A = {42, 17, 8, 93, 26, 55, 11, 68, 34, 5, 79, 21}. The search key is 68. The assignment also compares these methods with efficient alternatives including Merge Sort, a hybrid Merge/Insertion sorting method, and Binary Search.

Input: A = {42, 17, 8, 93, 26, 55, 11, 68, 34, 5, 79, 21}

Search key: K = 68

Expected sorted output: {5, 8, 11, 17, 21, 26, 34, 42, 55, 68, 79, 93}

Sequential Search result: 68 is at position 8 using 1-based indexing.

2. Selection Sort

Selection Sort is a brute-force sorting technique. During each pass, the algorithm searches the remaining unsorted portion for the smallest element and places it at the beginning of that portion.

2.1 Pseudocode

```
SELECTION_SORT(A, n)
```

```
for i = 0 to n-2
    min_index = i
    for j = i+1 to n-1
        if A[j] < A[min_index]
            min_index = j
    if min_index != i
        swap A[i] and A[min_index]
return A
```

2.2 Step-by-Step Execution

Pass	Smallest Element	Array After Pass
1	5	5 17 8 93 26 55 11 68 34 42 79 21
2	8	5 8 17 93 26 55 11 68 34 42 79 21
3	11	5 8 11 93 26 55 17 68 34 42 79 21
4	17	5 8 11 17 26 55 93 68 34 42 79 21
5	21	5 8 11 17 21 55 93 68 34 42 79 26
6	26	5 8 11 17 21 26 93 68 34 42 79 55
7	34	5 8 11 17 21 26 34 68 93 42 79 55
8	42	5 8 11 17 21 26 34 42 93 68 79 55
9	55	5 8 11 17 21 26 34 42 55 68 79 93
10	68	5 8 11 17 21 26 34 42 55 68 79 93
11	79	5 8 11 17 21 26 34 42 55 68 79 93

2.3 Comparison Count

For n elements:

$C(n) = (n-1) + (n-2) + ... + 1 = n(n-1)/2$

For n = 12:

$C(12) = 12 \times 11 / 2 = 66$ comparisons.

Therefore, Selection Sort performs exactly 66 comparisons for this input size.

2.4 Swaps and Data Movements

With the implementation used in this report, a swap is performed only when the minimum element is not already in the current position. The given input requires 9 swaps. Thus, comparisons and swaps must not be confused: there are 66 comparisons but only 9 swaps.

3. Sequential Search

Sequential Search checks elements one by one from the beginning of the array until the required key is found or the end of the array is reached.

3.1 Pseudocode

```
SEQUENTIAL_SEARCH(A, n, key)
```

```
for i = 0 to n-1
    if A[i] == key
        return i
return -1
```

3.2 Search for Key 68

Comparison	Element	Key	Result
1	42	68	Not equal
2	17	68	Not equal
3	8	68	Not equal
4	93	68	Not equal
5	26	68	Not equal
6	55	68	Not equal
7	11	68	Not equal
8	68	68	Found

The key 68 is the 8th element, so the search requires 8 comparisons.

4. Complexity Analysis

Algorithm	Best Case	Average Case	Worst Case
Selection Sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$
Sequential Search	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Merge Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$
Binary Search	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$

Selection Sort has $\Omega(n^2)$, $\Theta(n^2)$, and $O(n^2)$ behavior because it performs the same number of comparisons for a given n regardless of input arrangement. Sequential Search has $\Omega(1)$ best-case, $\Theta(n)$ average-case, and $O(n)$ worst-case behavior.

5. Efficient Alternatives

5.1 Merge Sort

Merge Sort follows the divide-and-conquer strategy. It divides the array into halves, recursively sorts the halves, and merges the sorted results. Its recurrence is $T(n) = 2T(n/2) + O(n)$, giving $\Theta(n \log n)$ time complexity.

5.2 Hybrid Merge/Insertion Sort

The hybrid implementation uses Merge Sort for large partitions and Insertion Sort when a partition contains at most 32 elements. This threshold reduces recursion overhead for small partitions.

5.3 Binary Search

Binary Search repeatedly divides a sorted search range into two halves. It requires the data to be sorted and has $O(\log n)$ worst-case search complexity. For one million elements, $\log_2(1{,}000{,}000)$ is approximately 19.93, so the number of comparisons is on the order of 20.

6. Real-World Dataset and Experimental Methodology

The proposed experiment uses the NYC Taxi & Limousine Commission (TLC) Trip Record Data. The numerical attribute selected is trip_distance. The official TLC dataset contains millions of real-world taxi trip records and is suitable for evaluating different input sizes.

Dataset source: NYC Taxi & Limousine Commission Trip Record Data

Attribute used: trip_distance

Preprocessing: remove missing values, remove invalid negative distances, convert to numeric values.

Input sizes: 1,000; 10,000; 100,000; and 1,000,000 records.

For a fair comparison, the same dataset subsets and computational environment should be used for all algorithms. Selection Sort is normally stopped at smaller sizes because its quadratic operation count makes one-million-element execution impractical.

7. Experimental Environment

Parameter	**Configuration**
Programming Language	Python 3
IDE	Visual Studio Code
Operating System	macOS
Dataset	NYC TLC Trip Record Data
Data Format	Parquet
Attribute	trip_distance
Timing	time.perf_counter()
Libraries	Pandas, PyArrow, Matplotlib
Hybrid threshold	32 elements

8. Theoretical Operation Counts for Selection Sort

Input Size (n)	**Comparisons n(n−1)/2**
1000	499,500
10000	49,995,000
100000	4,999,950,000
1000000	499,999,500,000

For n = 1,000,000, Selection Sort requires 499,999,500,000 comparisons, approximately 5 × 10^11. This explains why direct execution at this scale is impractical on a typical personal computer.

9. Expected Experimental Comparison

Input Size	**Selection Sort**	**Merge Sort**	**Hybrid Sort**
1000	Feasible	Fast	Fast
10000	Slower	Fast	Fast
100000	Very slow	Fast	Fast

1000000	Impractical; theoretical count reported	Practical	Practical

Actual execution times must be obtained by running the supplied Python program. They should not be invented because timing depends on hardware, operating system, Python version and system load.

10. Complete Python Implementation

```
import time

import pandas as pd

import matplotlib.pyplot as plt



def selection_sort(arr):

    a = arr.copy()

    n = len(a)

    comparisons = 0

    swaps = 0



    for i in range(n - 1):

        min_index = i

        for j in range(i + 1, n):

            comparisons += 1

            if a[j] < a[min_index]:

                min_index = j



        if min_index != i:

            a[i], a[min_index] = a[min_index], a[i]

            swaps += 1



    return a, comparisons, swaps





def merge_sort(arr):
```

```
    comparisons = [0]

    def merge(left, right):
        result = []
        i = j = 0

        while i < len(left) and j < len(right):
            comparisons[0] += 1
            if left[i] <= right[j]:
                result.append(left[i])
                i += 1
            else:
                result.append(right[j])
                j += 1

        result.extend(left[i:])
        result.extend(right[j:])
        return result

    def sort(a):
        if len(a) <= 1:
            return a
        mid = len(a) // 2
        return merge(sort(a[:mid]), sort(a[mid:]))

    return sort(arr.copy()), comparisons[0]


def hybrid_sort(arr, threshold=32):
    comparisons = [0]
```

```
def insertion_sort(a, left, right):
    for i in range(left + 1, right + 1):
        key = a[i]
        j = i - 1
        while j >= left:
            comparisons[0] += 1
            if a[j] > key:
                a[j + 1] = a[j]
                j -= 1
            else:
                break
        a[j + 1] = key


def merge(a, left, mid, right):
    temp = []
    i, j = left, mid + 1


    while i <= mid and j <= right:
        comparisons[0] += 1
        if a[i] <= a[j]:
            temp.append(a[i])
            i += 1
        else:
            temp.append(a[j])
            j += 1


    temp.extend(a[i:mid + 1])
    temp.extend(a[j:right + 1])


    for k, value in enumerate(temp):
        a[left + k] = value
```

```
    def sort(a, left, right):
        if left >= right:
            return

        if right - left + 1 <= threshold:
            insertion_sort(a, left, right)
            return

        mid = (left + right) // 2
        sort(a, left, mid)
        sort(a, mid + 1, right)
        merge(a, left, mid, right)

    result = arr.copy()
    if len(result) > 1:
        sort(result, 0, len(result) - 1)
    return result, comparisons[0]


def sequential_search(arr, key):
    comparisons = 0
    for i, value in enumerate(arr):
        comparisons += 1
        if value == key:
            return i, comparisons
    return -1, comparisons


def binary_search(arr, key):
    left, right = 0, len(arr) - 1
```

```
    comparisons = 0

    while left <= right:
        mid = (left + right) // 2
        comparisons += 1

        if arr[mid] == key:
            return mid, comparisons
        elif arr[mid] < key:
            left = mid + 1
        else:
            right = mid - 1

    return -1, comparisons



def load_dataset(file_path):
    df = pd.read_parquet(file_path)
    data = df["trip_distance"].dropna()
    data = data[data >= 0]
    return data.astype(float).tolist()



if __name__ == "__main__":

    example = [42, 17, 8, 93, 26, 55, 11, 68, 34, 5, 79, 21]

    print("Original:", example)

    sorted_array, comparisons, swaps = selection_sort(example)
```

```
print("Sorted:", sorted_array)
print("Selection comparisons:", comparisons)
print("Selection swaps:", swaps)

index, search_comparisons = sequential_search(example, 68)
print("68 position:", index + 1)
print("Sequential comparisons:", search_comparisons)

# For the large experiment, download an official TLC
# Parquet file and set its filename below.
file_path = "yellow_tripdata.parquet"
data = load_dataset(file_path)

sizes = [1000, 10000, 100000]

results = []

for n in sizes:
    arr = data[:n]

    start = time.perf_counter()
    _, sel_comp, sel_swaps = selection_sort(arr)
    sel_time = time.perf_counter() - start

    start = time.perf_counter()
    _, merge_comp = merge_sort(arr)
    merge_time = time.perf_counter() - start

    start = time.perf_counter()
    _, hybrid_comp = hybrid_sort(arr, threshold=32)
    hybrid_time = time.perf_counter() - start
```

```
    results.append({
        "n": n,
        "Selection Time": sel_time,
        "Selection Comparisons": sel_comp,
        "Selection Swaps": sel_swaps,
        "Merge Time": merge_time,
        "Merge Comparisons": merge_comp,
        "Hybrid Time": hybrid_time,
        "Hybrid Comparisons": hybrid_comp
    })

results_df = pd.DataFrame(results)
results_df.to_csv("sorting_results.csv", index=False)

plt.figure(figsize=(10, 6))
plt.plot(results_df["n"], results_df["Selection Time"],
         marker="o", label="Selection Sort")
plt.plot(results_df["n"], results_df["Merge Time"],
         marker="o", label="Merge Sort")
plt.plot(results_df["n"], results_df["Hybrid Time"],
         marker="o", label="Hybrid Sort")

plt.xlabel("Input Size")
plt.ylabel("Execution Time (seconds)")
plt.title("Sorting Algorithm Performance")
plt.xscale("log")
plt.grid(True)
plt.legend()
plt.savefig("sorting_execution_time.png", dpi=300)
plt.show()
```

11. Installation and Execution

Install the required packages in the VS Code terminal:

```
python3 -m pip install pandas pyarrow matplotlib
```

Then place the downloaded TLC Parquet file in the same folder as the Python program and run:

```
python3 daa_assignment.py
```

12. Results and Validation

The program generates sorting_results.csv and sorting_execution_time.png. The CSV file should be used to prepare the final experimental tables. Screenshots of the terminal output and graphs should be included in the report as evidence of tool usage.

Input Size	Selection Time	Selection Comparisons	Selection Swaps	Merge Time	Hybrid Time
1,000	Measure	499,500	Measure	Measure	Measure
10,000	Measure	49,995,000	Measure	Measure	Measure
100,000	Measure	4,999,950,000	Measure	Measure	Measure
1,000,000	Not executed	499,999,500,000	—	Measure	Measure

13. Comparative Analysis and Engineering Decision

Selection Sort and Sequential Search are attractive for small inputs because they are easy to understand and implement and require little algorithmic infrastructure. However, their brute-force nature becomes a major limitation as n increases. Selection Sort grows quadratically, while Sequential Search grows linearly in the worst case.

For large sorting problems, Merge Sort or a suitable hybrid sorting method is preferable. The hybrid method is selected because it combines divide-and-conquer sorting for large partitions with the low overhead of Insertion Sort for small partitions. For repeated searches, Binary Search is preferable when the data can be sorted in advance.

14. Broader Considerations

Sustainability: Efficient algorithms can reduce CPU time and unnecessary computation, which may lower energy consumption when large-scale processing is repeated.

Economic impact: Faster algorithms can reduce processing time and computing-resource requirements.

SDG 4 – Quality Education: The assignment develops analytical thinking, programming, mathematical reasoning and computational problem-solving skills.

SDG 9 – Industry, Innovation and Infrastructure: Algorithmic efficiency is important for building scalable computational systems that process growing volumes of data.

15. Limitations

- Selection Sort is impractical for one million elements because of its quadratic operation count.
- Execution times depend on hardware, operating system, Python version and system load.
- Python interpreter and memory-management overhead can affect measured times.
- Binary Search requires sorted data, so preprocessing may add a one-time sorting cost.
- Real-world data requires preprocessing for missing or invalid values.

16. Conclusion

The assignment demonstrates the application of brute-force algorithms through Selection Sort and Sequential Search. For the given 12-element array, Selection Sort produces the sorted sequence {5, 8, 11, 17, 21, 26, 34, 42, 55, 68, 79, 93}, performs 66 comparisons and 9 swaps under the specified implementation. Sequential Search finds 68 at position 8 after 8 comparisons.

The theoretical and experimental analysis shows that Selection Sort has $\Theta(n^2)$ time complexity and Sequential Search has $\Theta(n)$ average/worst-case behavior. Merge Sort and the hybrid sorting approach scale more effectively for large datasets, while Binary Search provides logarithmic search time when sorted data is available. The experiment therefore demonstrates why algorithm selection must consider both implementation simplicity and scalability.

17. Student Reflection

Through this assignment, I gained a better understanding of how algorithmic complexity affects the practical performance of a program. Selection Sort and Sequential Search are simple to implement, but their operation counts show that simple algorithms can become inefficient as input size increases.

I learned that Selection Sort performs $n(n-1)/2$ comparisons and that comparisons must be distinguished from data movements or swaps. I also learned how divide-and-conquer algorithms such as Merge Sort provide better scalability and how a hybrid method can combine different techniques for practical performance.

Working with a real-world dataset introduced additional challenges such as data cleaning, missing values, input-size selection and timing large experiments. If additional resources were available, I would compare more hybrid algorithms such as IntroSort and TimSort, repeat experiments to obtain averages and standard deviations, and evaluate the algorithms on different hardware platforms.

18. GitHub Upload Structure

```
CSA0603-DAA-Assignment/
│
├── README.md
├── code/
│   └── daa_assignment.py
├── dataset/
│   └── README.md
```

```
├── results/
│   ├── sorting_results.csv
│   ├── search_results.csv
│   ├── sorting_execution_time.png
│   └── search_execution_time.png
├── report/
│   └── DAA_Assignment.docx
└── screenshots/
    ├── selection_sort_output.png
    ├── sequential_search_output.png
    └── experiment_output.png
```

19. References

1. NYC Taxi & Limousine Commission. TLC Trip Record Data. Official NYC Government dataset portal.
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms. MIT Press.
3. Python Software Foundation. Python Documentation.
4. Pandas Documentation. Python Data Analysis Library.
5. Matplotlib Documentation. Visualization with Python.